

# workable-items – § . . SQLite-SSoT Go binary

---

**Revision:** 4 **Last modified:** 2026-08-25T00:00:00Z **Description:** Authoritative source-of-truth registry for every workable item in every consuming project. Bidirectional regen between SQLite DB and Markdown trackers per Constitution §11.4.93.

## Overview

---

Per Constitution §11.4.93, every consuming project's workable-items inventory (Issues.md / Fixed.md / Issues\_Summary.md / Fixed\_Summary.md / Status.md fleet / CONTINUATION.md §3) is reconstructible from this DB-backed registry. The DB is the **authoritative** source; all Markdown / HTML / PDF surfaces are generator output.

Sync drift is mechanically impossible because every regeneration starts from the DB.

# Layout

| Path                                    | Purpose                                                                                                     |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>schema.sql</code>                 | DDL — 6 tables: items + item_history + obsolete_details + operator_block_details + firebase_metadata + meta |
| <code>cmd/workable-items/main.go</code> | Go binary entrypoint (Phase 2 scaffold)                                                                     |
| <code>pkg/itemsdb/</code>               | SQLite access layer (Phase 2 scaffold)                                                                      |
| <code>pkg/mdparser/</code>              | Markdown → DB parser (Phase 3 scaffold)                                                                     |
| <code>pkg/mdrender/</code>              | DB → Markdown renderer (Phase 4 scaffold)                                                                   |
| <code>tests/</code>                     | Unit + integration + stress + chaos tests per §11.4.85                                                      |

## Subcommands (implemented)

All subcommands below are implemented and tested against a real SQLite DB (no mocks). `--db <path>` is required by every subcommand.

```

workable-items sync md-to-db --db <p> [--issues <p>] [--fixed <p>]
                                # parse Issues.md+Fixed.md → upsert DB
workable-items sync db-to-md --db <p> [--out-issues <p>] [--out-fixed <p>]
                                # regen MD docs from DB (byte-
identical round-trip)
workable-items diff --db <p> [--issues <p>] [--fixed <p>] [--partial-
scope] | --db-only
                                # show DB vs MD divergence (CI gate).
                                # REFUSES rather than emit a verdict

it cannot
                                # support: with NO Markdown path

(BOB-155), and
                                # when the supplied path(s) do not

account for
                                # every DB item's current_location

(BOB-186) —
                                # e.g. --issues alone while the DB

holds Fixed
                                # rows, which previously reported "in
sync"
                                # having compared none of them.
                                # --db-only      : DB-internal checks

only, reads
                                #
                                # no Markdown (verdict
says so).
                                # --partial-scope: compare only the

tracker(s)
                                #
                                # supplied; the

verdict names the
                                #
                                # locations it did NOT

cover and
                                #
                                # never claims a full
sync.
workable-items validate --db <p> # invariant + schema sanity (pre-build
gate)
workable-items add <type> <severity> --db <p> --title <T> --description
<D> [--id <id>] [--prefix <P>]
                                # create a new Queued item in Issues;

--id
                                # auto-generated as <PREFIX>-NNN when

absent
                                # (PREFIX defaults to WIT). §11.4.91

floor
                                # enforced on --description at entry.
workable-items update --id <ID> --db <p> [--title|--severity|--

```

```

description|--type|--status|--created-by|--assigned-to ...] [--location
Issues|Fixed]

# mutate ONLY the explicitly-set
fields on an

# existing item; regenerate body_md;
append an

# item_history 'Updated' row
 (§11.4.34). Rejects

# unknown ids, no-field calls,
 §11.4.91-short desc.
workable-items reopen --id <ID> --db <p> --why <reason> --who <AI|User> --
when <ISO> --incident <p> [--location Issues|Fixed]

# set Status=Reopened with the four
 §11.4.34

# attribution facts (By/On/Reason/
Evidence). --why

# drawn from the closed set: test-
failed |

# manual-testing-detected | captured-
evidence-

# contradicts | end-user-report |
cycle-re-

# discovered | design-reconsidered.
ALL four

# mandatory (a reopen is a §11.4.7
demotion).
workable-items block --id <ID> --db <p> --details <WHAT> [--why <T>] [--
unblock <T>] [--who <T>] [--location Issues|Fixed]

# set Status=Operator-blocked + an
# operator_block_details row
 (§11.4.21). --details

# (the WHAT) is mandatory and non-
empty.
workable-items close <atm-id> --db <p> --status <fixed|implemented|
completed|obsolete> --evidence <p>

# atomic Issues→Fixed move (§11.4.19);
--evidence

# mandatory (§11.4.5/§11.4.90);
records item_history.
workable-items report --db <p> [--by-type|--by-status|--by-severity|--by-
assigned|--by-creator|--obsolete-audit]

# read-only grouped tally; --obsolete-
audit lists

# Obsolete items + flags missing
 §11.4.90 details.
workable-items export --db <p> [--out-dir <d>] [--no-formats] [--out-

```

```

issues <p>] [--out-fixed <p>]
                                # regenerate Issues.md + Fixed.md
(byte-identical
                                # round-trip) + Issues_Summary.md
 (§11.4.12 open-
                                # only Type×Status tally) +
Fixed_Summary.md
                                # (§11.4.53 closed-only) + §11.4.65
HTML/PDF/DOCX
                                # siblings via pandoc (--pdf-
engine=weasyprint).
                                # Sibling steps are exec.LookPath-
gated: when
                                # pandoc/weasyprint is absent the .md
is still
                                # written + an honest message printed
– NO fake
                                # sibling is fabricated (§6.J/
§11.4.6).

```

Positional and flag arguments may be given in either order (e.g. both `add Bug Critical --db x ...` and `add --db x ... Bug Critical` are accepted).

`add` , `update` , `reopen` , `block` , and `close` all mutate the DB *and* keep the §11.4.93 byte-identical round-trip intact: they generate / regenerate / move the canonical `## <ID> – <title>` item block + its `doc_segments` entry so a subsequent `sync db-to-md` regenerates a well-formed, re-parseable tracker (add is a stable fixed point; close round-trips and re-validates; update/reopen/ block regenerate `body_md` with the new status + any §11.4.34/§11.4.21 detail line embedded in the heading-adjacent meta block).

# Anti-bluff coverage (mandatory per § . . + § . . )

| Layer                          | Test                                                                                          |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| Unit                           | every CRUD operation + every schema invariant + closed-set validators                         |
| Integration                    | full MD → DB → MD round-trip byte-identical (modulo whitespace tolerance)                     |
| Stress per §11.4.85            | 1000-row insert; 10 concurrent writers; sustained read throughput                             |
| Chaos per §11.4.85             | mid-write SIGKILL; corrupt-DB recovery; disk-full; FD exhaustion                              |
| Paired §1.1 mutation           | strip CRUD function → unit test FAILs                                                         |
| HelixQA Challenge <sup>6</sup> | CME-WORKABLE-ITEMS-001 <sub>1 1</sub> exercises end-to-end DB → MD → DB → MD <sub>4 9 3</sub> |

## -phase migration per § . . (consuming-project work)

1. **Phase 1** — Issues entry §SLA filed, scope-locked.
2. **Phase 2** — Go binary scaffold + DDL committed (this PWU).
3. **Phase 3** — sync md-to-db lands + captures initial migration.
4. **Phase 4** — sync db-to-md lands + byte-identical round-trip CI gate.
5. **Phase 5** — Existing bash generators ( generate\_issues\_summary.sh etc.) become Go-binary shims.
6. **Phase 6** — Legacy text-direct edits prohibited (pre-commit hook).

## Composes with constitution anchors

---

§11.4 / §11.4.12 / §11.4.15 / §11.4.16 / §11.4.17 / §11.4.19 / §11.4.21 / §11.4.27 /  
§11.4.30 / §11.4.33 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.50 /  
§11.4.52 / §11.4.53 / §11.4.54 / §11.4.55 / §11.4.56 / §11.4.57 / §11.4.58 / §11.4.60 /  
§11.4.65 / §11.4.74 / §11.4.77 / §11.4.83 / §11.4.85 / §11.4.86 / §11.4.87 / §11.4.89 /  
§11.4.90 / §11.4.91 / §11.4.92.

## Canonical authority

---

`../../Constitution.md` §11.4.93.